1. In the code above what is the meaning of remainder? What exactly is this piece of code doing? What is the significance or the function of double underscore in pipeline construction

The referenced code is focused on creating an instance of the ColumnTransformer class from scikitlearn.compose. It is intended to be used with the housing dataframe. The housing dataframe has nine columns of “feature” data we want to use to predict median_house_value. The ColumnTransformer is instantiated with a set of make_pipeline functions in a list of triples.

The remainder parameter sets a make_pipeline function call for any feature not consumed in the first parameter of the call. This is obviously done by magic. Ok, maybe not magic, but a complicated process behind the api.

Here we’ve defined the instantiation call and instantiated it:

preprocessing = ColumnTransformer([etc…

Next:

We call model fit – processing.fit_transform(housing) on the preprocessing object with the housing dataframe and again magic happens.

Part of the magic is that the features are now renamed based on the configuration passed in the instantiation with transformed data.

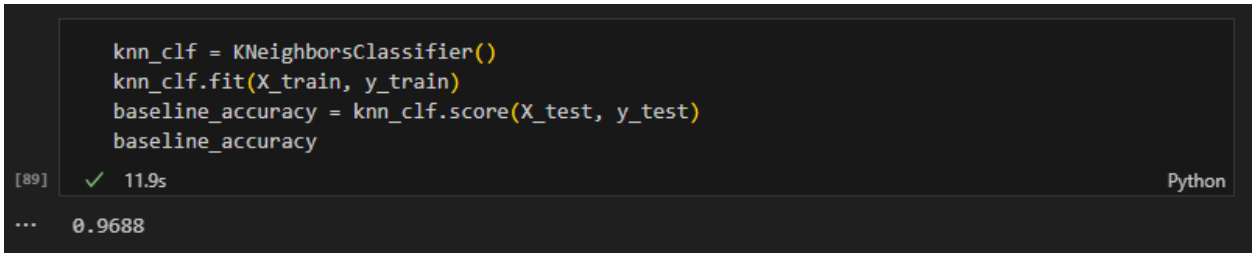
preprocessing.get_feature_names_out() outputs a list of those names

they have the format – ‘transformer name (from config)” & __ & “feature name”

This convention with the double __ is useful in figuring out later what’s what how it got here, but also when referencing when running a Grid search.

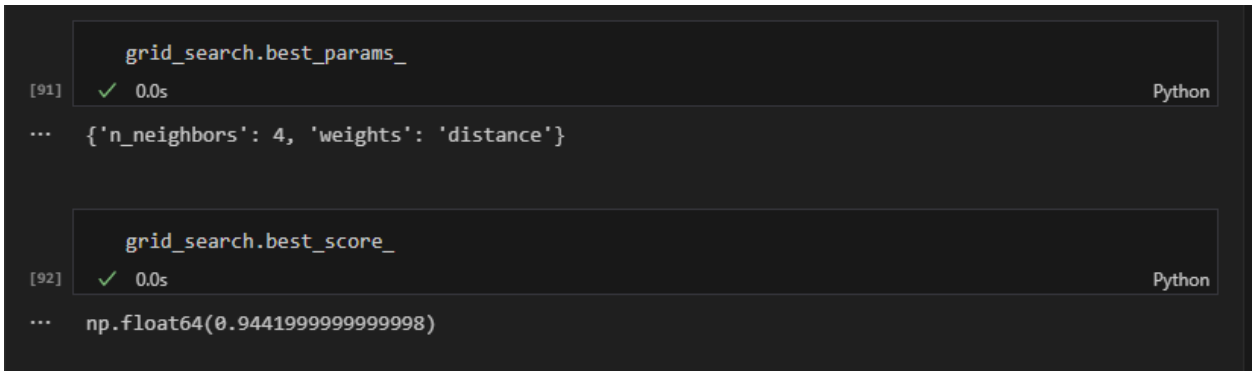
2. Try to build a classifier for the MNIST dataset that achieves over 97% accuracy on the test set. Hint: the KNeighborsClassifier works quite well for this task; you just need to find good hyperparameter values (try a grid search on the weights and n_neighbors hyperparameters)

Here the KNeighborsClassifier baseline score is 96.88%



We do a GridSearchCV over weights and n_neighbors as defined in param_grid:

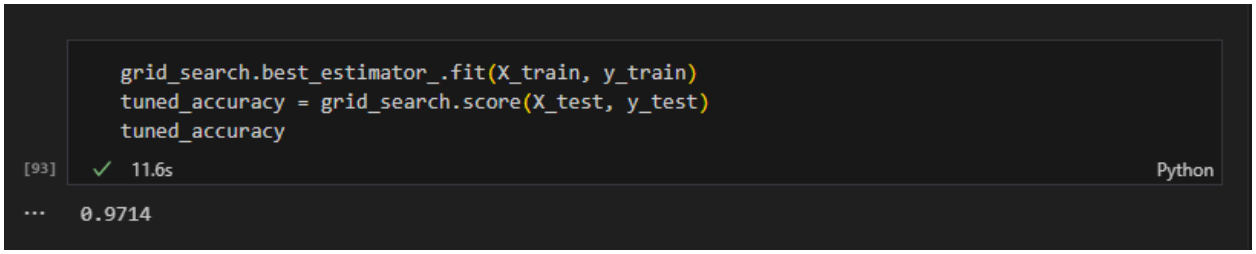
Choose best params, and best score for restricted (10k obs) set is 94.42%



Best n_neighbors = 4 and weights is distance

But:

If we use the best_estimator fit on the whole set:



We get 97.14%

3. Tackle the Titanic dataset. A great place to start is on Kaggle. Alternatively, you can download the data from https://homl.info/titanic.tgz and unzip this tarball like you did for the housing data in Chapter 2. This will give you two CSV les, train.csv and test.csv, which you can load using pandas.read.csv(). The goal is to train a classifier that can predict the Survived column based on the other columns

I've downloaded and expanded the Titanic data to a local folder.

```
import pandas as pd

# Read the Titanic training dataset
train_df = pd.read_csv('../lectures/HandsOn_Chapter3/datasets/titanic/train.csv')

# Display basic information about the dataset
print(f"Dataset shape: {train_df.shape}")
print(f"\nFirst few rows:")
train_df.head()
```

[1] ✓ 0.6s Open 'train_df' in Data Wrangler Python

```
Dataset shape: (891, 12)

First few rows:
```

	# PassengerId	# Survived	# Pclass	Name	Se
0	1	0	3	Braund, Mr. Owen Harris	male
1	2	1	1	Cumings, Mrs. John Bradley (Florenc	femal
2	3	1	3	Heikkinen, Miss. Laina	femal
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily Ma	femal
4	5	0	3	Allen, Mr. William Henry	male

There are 891 rows and 12 columns in train_df

```
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
```

We see a few missing Embarked values, some missing Age values, and a lot of missing Cabin values.

Survived (0, 1) is the “Y” we are looking to predict.

Overall 61.6% percent did not survive (0), and 38.4% survived (1).

I’m throwing out Ticket – looks like random strings, Name (not useful in the regression), and PassengerId (looks like an index).

Features used:

'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked'

Setting

X = train_df[features_used]

Y = train_df[‘Survived’]

For Sex [‘male’, ‘female’], I reset them to 0, 1 respectively

For Embarked[‘C’, ‘Q’, ‘S’], I fill the missing with S (most frequent) and One-Hot encode the field.

For Age, and Fare I fill the missing with the median

Split the data into training and test set, and apply transform

```
# Split data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Training set size: 712

Test set size: 179

Training set survival rate: 0.383

Test set survival rate: 0.385

Training Multiple Classifiers:

```python
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Initialize classifiers
classifiers = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM': SVC(random_state=42),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=5)
}
```

The best model was Decision Tree (Test Accuracy 81.5%)

```
Best Model: Decision Tree

============================================================

Classification Report:
                 precision    recall  f1-score   support

Did not survive       0.84      0.86      0.85       110
       Survived       0.77      0.74      0.76        69

       accuracy                           0.82       179
      macro avg       0.81      0.80      0.80       179
   weighted avg       0.81      0.82      0.81       179


Confusion Matrix:
[[95 15]
 [18 51]]

Confusion Matrix Interpretation:
True Negatives (correctly predicted did not survive): 95
False Positives (incorrectly predicted survived): 15
False Negatives (incorrectly predicted did not survive): 18
True Positives (correctly predicted survived): 51
```

Most important predictor was Sex

```
Feature Importances:
      Feature  Importance
1         Sex    0.316638
2         Age    0.252508
5        Fare    0.233430
0      Pclass    0.110281
7  Embarked_S    0.032651
4       Parch    0.029445
3       SibSp    0.019436
6  Embarked_Q    0.005611
```

Work in

https://github.com/McSavage/FIN220/blob/main/assignments/FIN220_HW3_Savage.ipynb